

자료구조

2분반

HW4

보고서

담당교수님 : 장형수 교수님

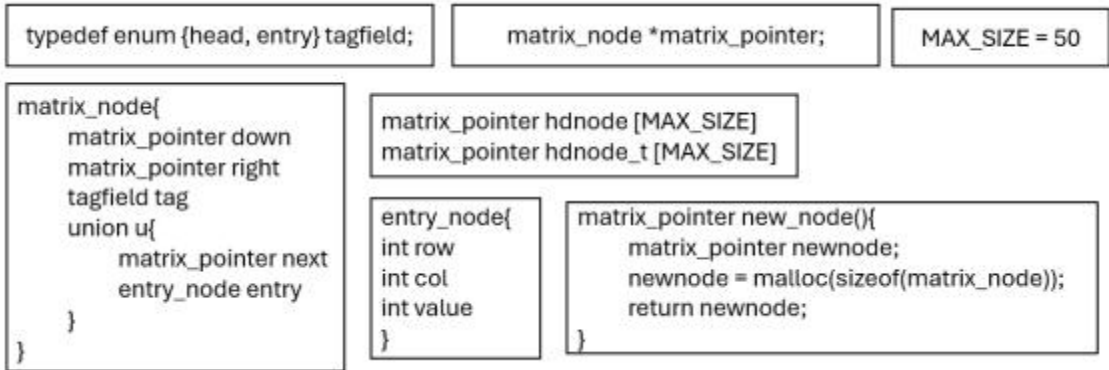
학과 : 컴퓨터공학과

학번: 20211562

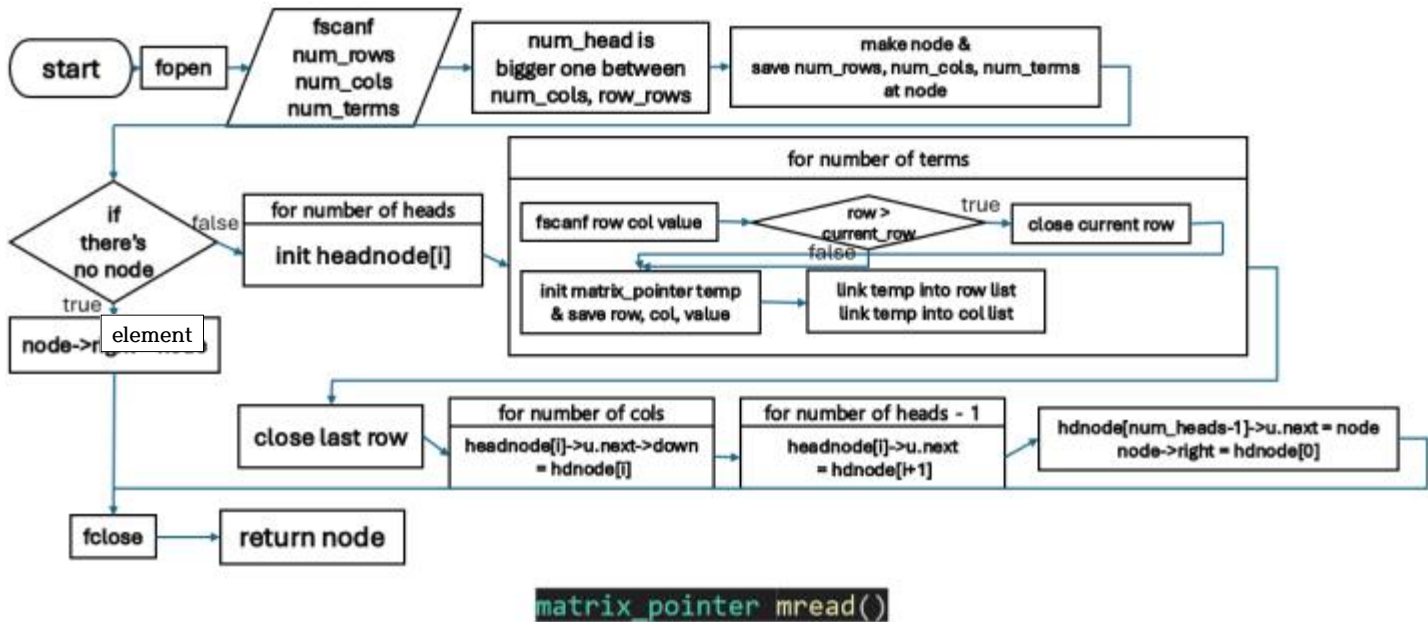
이름: 이동협

문제1

문제 1은 input.txt 파일을 통해 matrix를 입력받은 뒤 이를 matrix_pointer로 linked list로 저장한 뒤 이를 transpose하여 transposed된 matrix를 output.txt를 통해 출력하는 문제이다.



위의 code에서 동작하기 위해 필요한 data type과 matrix_pointer를 initialize하는 new_node() 함수를 구현한 것이다. hdnode는 입력받은 matrix를 저장하는 포인터이고 hdnode_t는 transpose한 matrix를 저장하는 포인터이다.



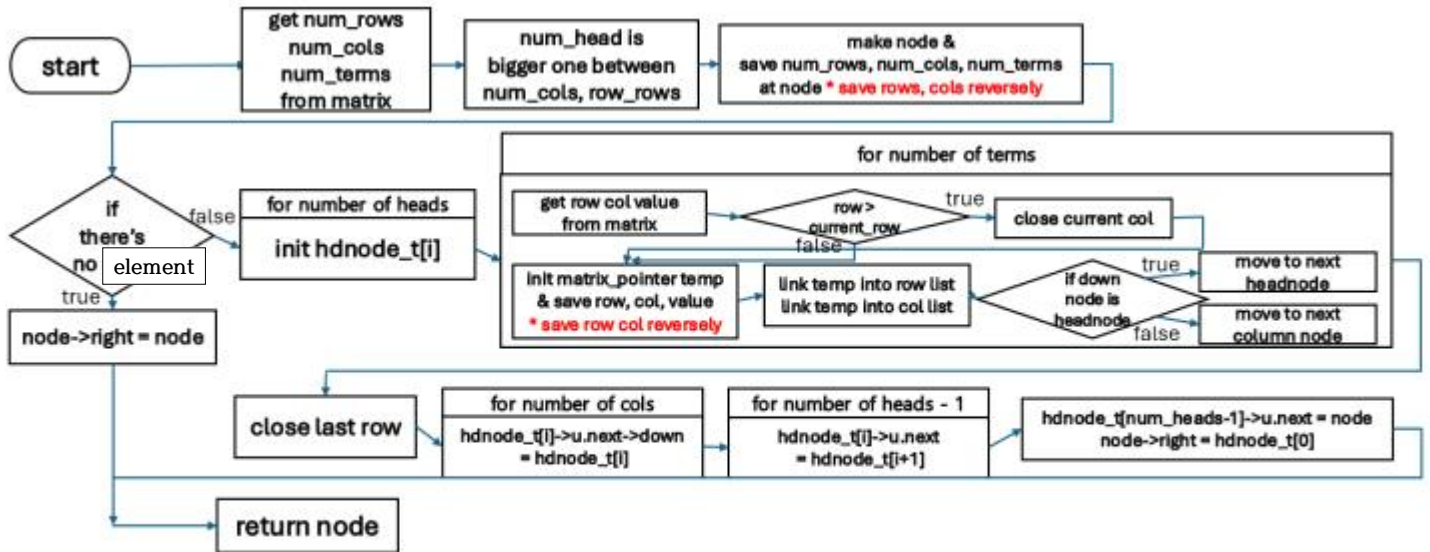
mread 함수는 input.txt를 열어 첫 줄에서 각 row, col, term의 개수를 읽고 row와 col중 더 많은 개수를 가지는 개수를 headnode의 개수로 저장한다. 이후 노드를 만들어 row, col, term의 개수를 저장한다.

이후 읽은 파일의 term 개수가 0이라면 node의 right에 node를 저장하고 파일을 닫고 끝낸다.

그렇지 않다면 head의 개수만큼 headnode를 만들어 initialize한다. 이후 파일의 term의 개수만큼 파일에서 한줄씩 row, col, value를 읽고 한 row씩 element를 matrix_pointer로 row, col list와 연결하고 읽은 row가 진행중이던 row보다 크다면 현재 row를 닫고 다음 row로 이동한다.

이 작업이 끝나면 연결을 진행했던 마지막 row를 닫고 col의 개수만큼 hdnode[0]부터 down으로 이동하며 각 hdnode들을 link한다. 이후 head의 개수 -1개 만큼 hdnode[0]부터 next로 이동하며 각 hdnode를 연결하고 마지막 hdnode는 node(각 개수를 저장한 node)를 연결한다. node의 right에는 hdnode[0]를 연결하여 모든 hdnode와 node가 연결되도록 한다.

이후 파일을 닫고 완성한 linked list의 matrix_pointer를 return한다.



```
matrix_pointer mtranspose(matrix_pointer matrix)
```

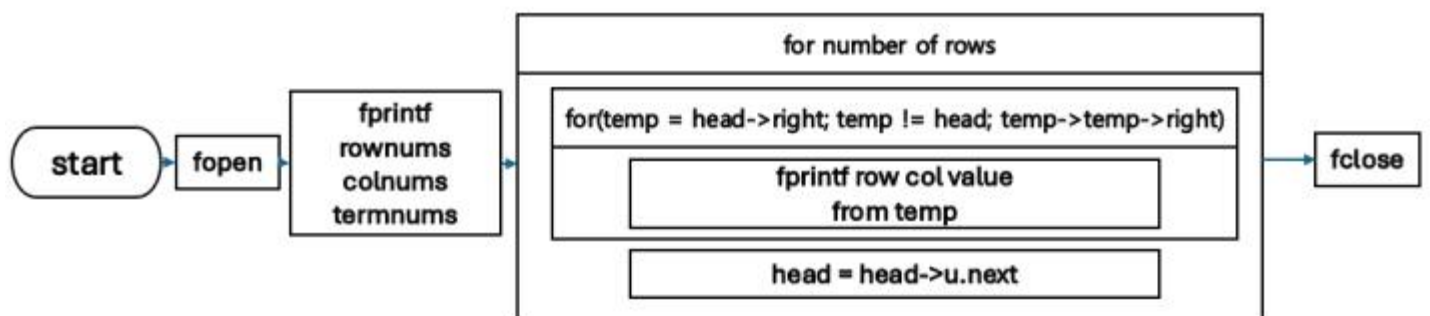
mtranspose 함수는 argument로 matrix_pointer를 받아 node에서 각 row, col, term의 개수를 읽고 row와 col중 더 많은 개수를 가지는 개수를 headnode의 개수로 저장한다. 이후 노드를 만들어 row, col, term의 개수를 저장한다. 이때 row와 col의 개수를 바꿔 저장한다.

이후 matrix의 term 개수가 0이라면 node의 right에 node를 저장하고 파일을 닫고 끝낸다.

그렇지 않다면 head의 개수만큼 headnode를 만들어 initialize한다. 이후 argument matrix의 term의 개수만큼 argument matrix에서 한줄씩 row, col, value를 읽고 matrix_pointer에 row와 col을 서로 바꿔 저장한다. 한 col씩 element를 matrix_pointer로 row, col list와 연결하고 argument matrix에서 읽은 row가 진행중이던 row보다 크다면 현재 row를 닫고 다음 row로 이동한다. transpose matrix에 하나씩 element를 연결하면서 argument matrix의 down node가 headnode라면 다음 헤드노드로 이동해 down node를 불러온다.

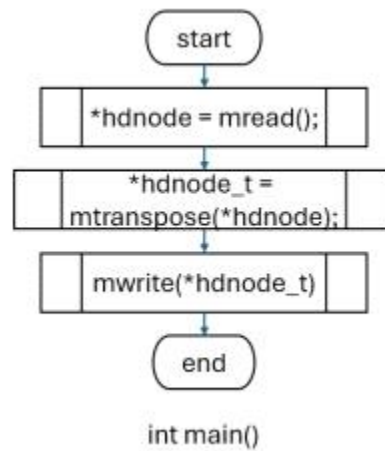
이 작업이 끝나면 연결을 진행했던 마지막 col를 닫고 col의 개수만큼 hdnode_t[0]부터 down으로 이동하며 각 hdnode들을 link한다. 이후 head의 개수 -1개 만큼 hdnode_t[0]부터 next로 이동하며 각 hdnode_t를 연결하고 마지막 hdnode_t는 node(각 개수를 저장한 node)를 연결한다. node의 right에는 hdnode_t[0]를 연결하여 모든 hdnode_t와 node가 연결되도록 한다.

이후 파일을 닫고 완성한 linked list의 matrix_pointer를 return한다.



```
void mwrite(matrix_pointer node)
```

mwrite 함수는 matrix 가 저장된 matrix_pointer를 argument로 받아 output.txt의 첫줄에 row, col, term의 개수를 쓴다. 이후 row의 개수만큼 각 열에서 matrix_pointer를 right으로 이동하면서 저장된 element의 row, col, value를 output.txt에 한줄씩 쓴다. 이 작업이 모두 끝나면 파일을 닫고 끝낸다.



main 함수에서는 mread를 통해 input.txt의 matrix를 linked list로 변환한다. 이후 이를 가리키는 matrix_pointer를 hdnode에 저장한다. 이후 mtranspose를 통해 hdnode의 matrix를 transpose하여 이 matrix를 가리키는 matrix_pointer를 hdnode_t에 저장한다. 이후 mwrite를 통해 hdnode_t가 가리키는 matrix를 output.txt에 저장한다.

```

cse20211562@cspro1: ~/structure/hw4
1 2 5 6
2 0 2 11
3 0 4 6
4 1 0 12
5 1 1 7
6 2 1 -4
7 3 3 -15

"input.txt" [noeol] 7L, 46C 1,1 All

cse20211562@cspro1:~/structure/hw4$ gcc 1.c
cse20211562@cspro1:~/structure/hw4$ ./a.out
cse20211562@cspro1:~/structure/hw4$ cat output.txt
5 4 6
0 1 12
1 1 7
1 2 -4
2 0 11
3 3 -15
4 0 6
cse20211562@cspro1:~/structure/hw4$
  
```

이는 제시된 출력 예시를 만족한다.

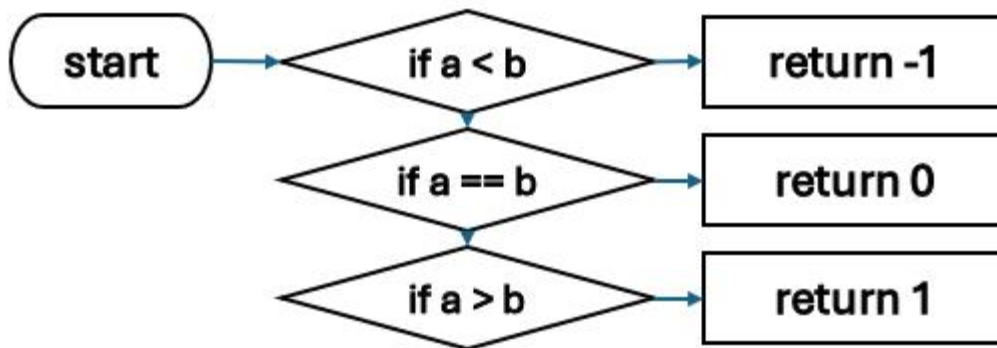
문제2

문제 2는 a.txt, b.txt에 저장된 두 polynomials의 곱셈의 결과값을 d.txt에 저장하는 문제이다. polynomial을 구현하기 위해 linkedlist를 사용하였다.

```
poly_node *poly_pointer;
```

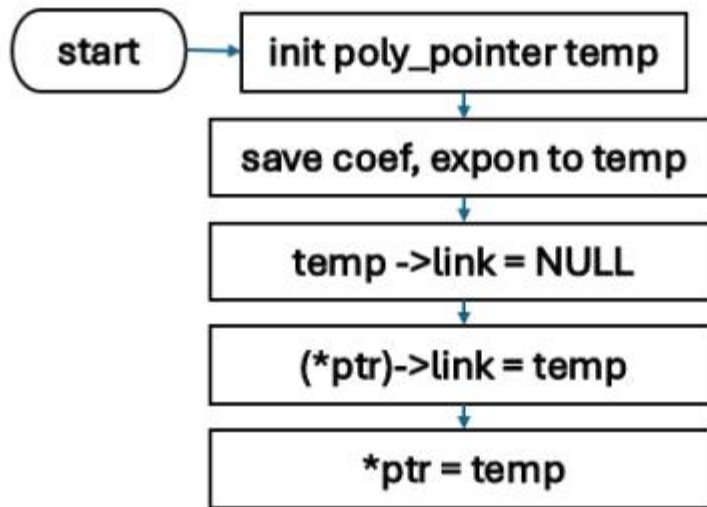
```
poly_node{  
    poly_pointer link  
    int num  
    int coef  
    int expon  
}
```

polynomial을 저장하기 위한 polynode와 이를 가리키는 polypointer이다. num에는 linked list의 element 개수가 저장되고 coef에는 element의 coefficient, expon에는 element의 exponential이 저장된다.



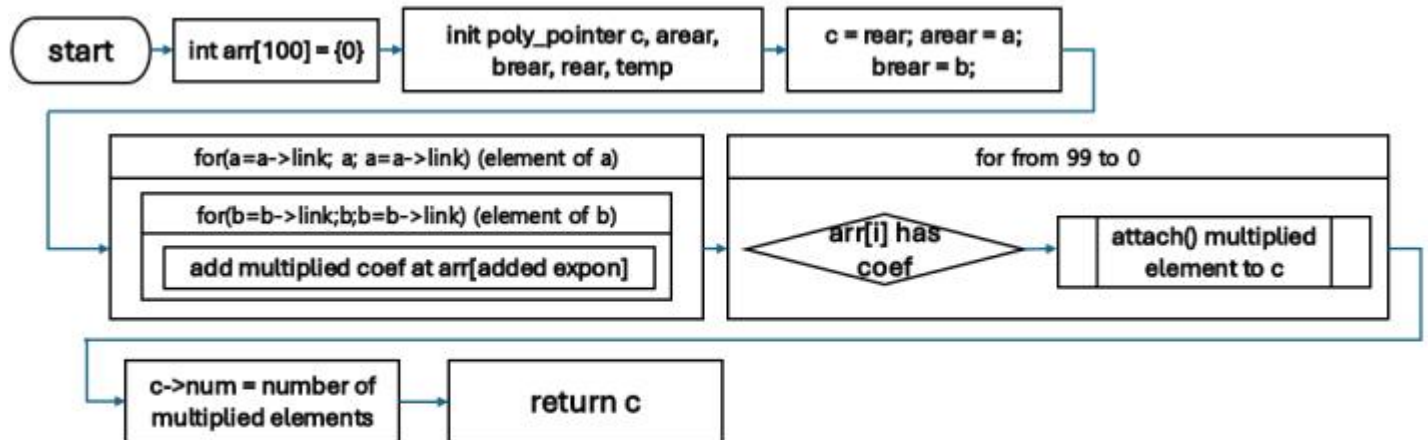
```
int COMPARE(int a, int b)
```

COMPARE 함수는 integer variable a와 b를 argument로 받아서 b가 더 크면 -1, 같으면 0, a가 더 크면 1을 return 한다.



```
void attach(int coefficient, int exponent, poly_pointer *ptr)
```

attach함수는 coefficient, exponential 값을 받아서 poly_pointer temp를 새로 declare해서 이에 저장한 후 polypointer ptr에 연결해준다.



```
poly_pointer pmult(poly_pointer a, poly_pointer b)
```

pmult 함수는 polypointer a, b를 받아서 두 polynomials의 곱을 리턴하는 것이다.

가장 먼저 이들을 곱한 polynomial의 element 개수를 곱해보기 전까지는 알 수 없기 때문에 각 차수의 coefficient 값을 저장할 배열 arr을 declare해준다. 이중 for loop으로 a와 b의 linked list의 각 element를 one by one으로 곱하여 곱한 expon번째 arr에 exponential 값을 더해준다. polynomials의 곱하기가 끝나면 내림차순으로 polynomial을 sort 하기 위해 arr을 역순으로 읽으면서 coefficient가 존재하는(0이 아닌) exponen의 값을 attach를 사용하여 리턴할 polypointer인 c에 연결해준다. c에 각 element를 연결하는 작업이 끝나면 c의 head에 element의 개수를 저장하고 c를 return해준다.


```
cse20211562@cspro1: ~/structure/hw4
1 1
2 1 5
3 6 3
4 2 2
5 4 1

"a.txt" [noel] 5L, 17C 1,1 All
```

```
cse20211562@cspro1: ~/structure/hw4
1 8
2 7 3
3 -1 1
4 2 0

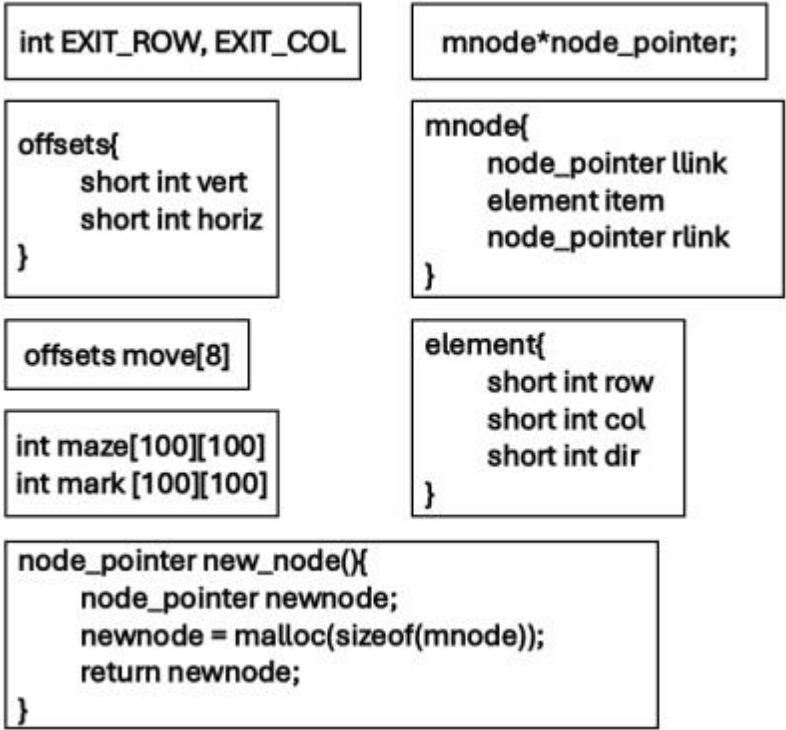
"b.txt" [noel] 4L, 14C 1,1 All
```

```
cse20211562@cspro1:~/structure/hw4$ gcc 2.c
cse20211562@cspro1:~/structure/hw4$ ./a.out
cse20211562@cspro1:~/structure/hw4$ cat d.txt
6
7 8
41 6
16 5
22 4
10 3
8 1
cse20211562@cspro1:~/structure/hw4$
```

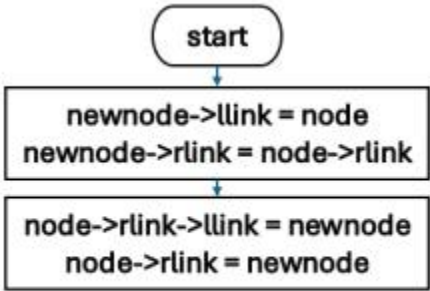
제시된 출력 예시를 만족한다.

문제3

문제 3은 maze.txt를 통해 maze의 모형을 읽고 maze의 (1,1)에서 (EXITROW-2, EXITCOL-2)까지 이어지는 path를 path.txt를 통해 출력하는 문제이다.

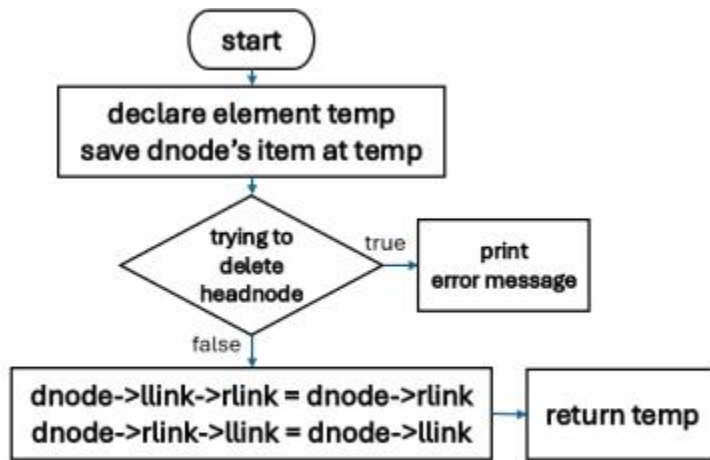


문제3을 해결하기 위해 사용한 struct와 전역변수들이다. EXIT_ROW와 EXIT_COL은 미로의 출구 좌표를 지정하기 위한 variable이며 offset은 move[8]을 통해 미로에서의 각 방향의 움직임에 대해 좌표가 어떻게 변화하는지를 저장하기 위한 struct이다. maze와 mark는 각각 미로의 형태와 미로를 빠져나가기 위해 이동한 자취를 저장하는 2D array다. mnode는 mazenode로 maze를 탈출하는 path를 doubly linked list로 저장하기 위한 struct이다. node_pointer는 mnode의 포인터이고 element는 이동에 대한 정보 row, col, dir을 저장하는 struct이다. new_node()는 새로운 node_pointer를 사용하기 위해 node_pointer를 initialize하고 return해주는 함수이다.



```
void dinser(node_pointer node, node_pointer newnode)
```

dinser함수는 doubly linked list의 node 오른쪽에 newnode를 연결하는 함수이다. newnode의 llink를 node로 바꾸고 rlink를 node가 rlink하던 노드로 바꾼다. 이후 node가 rlink하던 노드의 llink를 newnode로 바꾸고 node의 rlink를 newnode로 바꾸어 insert 작업을 마무리한다.

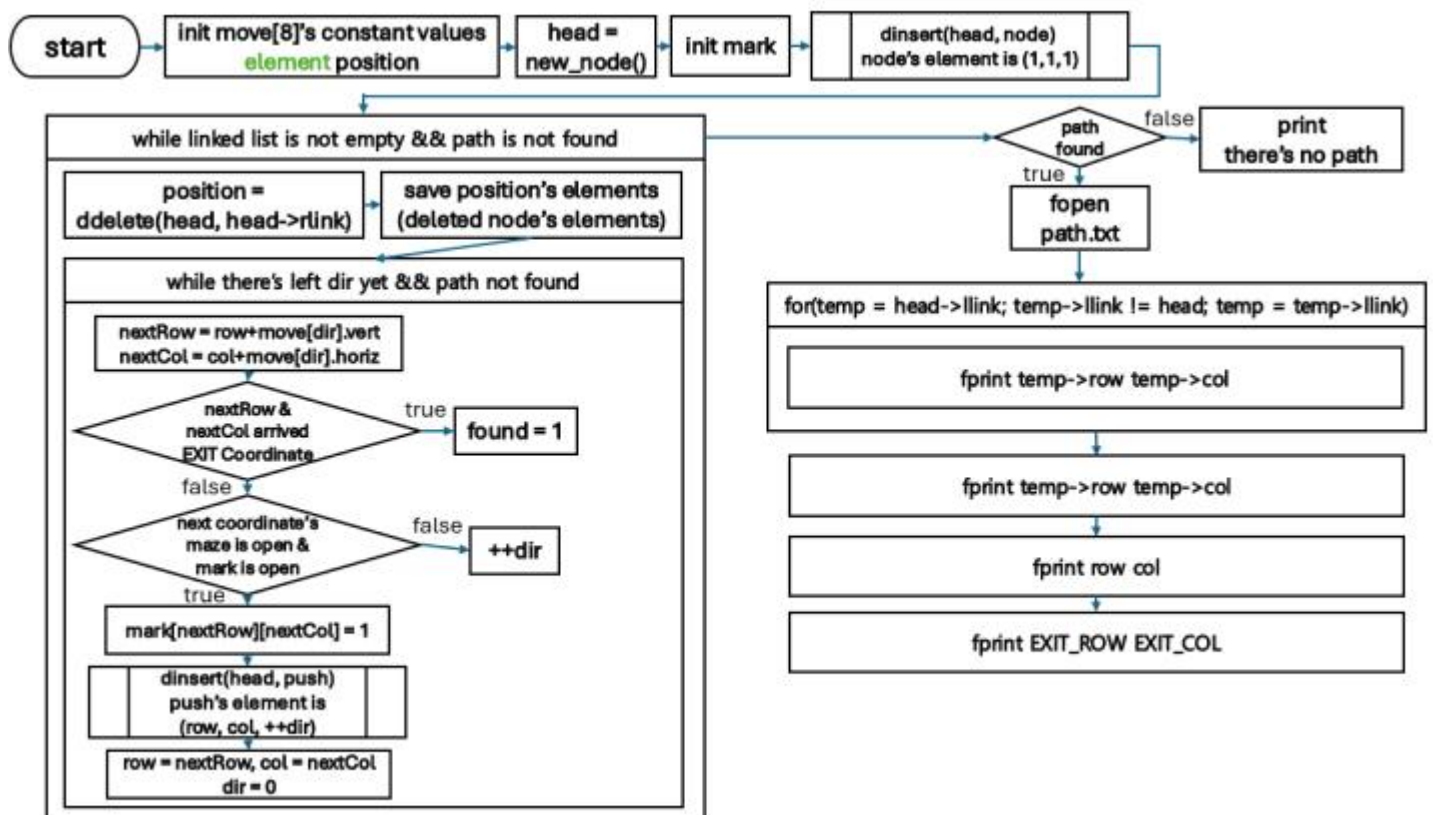


```
element ddelete(node_pointer node, node_pointer dnode)
```

ddelete함수는 node의 rlink 노드인 dnode를 doubly linked list에서 delete하면서 dnode가 저장하던 element 값을 return하는 함수이다.

가장 먼저 dnode의 element 값을 temp에 저장한다. 이후 삭제하려는 dnode가 head node라면 에러메세지를 출력하면서 끝낸다. head node를 지우려는 것이 아니라면 다음의 과정을 따른다.

먼저, dnode의 llink 노드의 rlink를 dnode의 rlink 노드로 바꾼다. 이후 dnode의 rlink 노드의 llink를 dnode의 llink 노드로 바꾼다. 이후 temp에 저장한 element 값을 return 한다.



```
void path(void)
```

path함수는 가장 먼저 maze 상에서 한 번의 이동 시에 사용하는 경우의 수를 저장하는 move[8]의 값을 일일이 저장해준다.(ex. 위로 이동할 때는 row가 1 줄고 col은 유지되고 하는 정보들) 이후 head node와 mark(초기 위치 설정)를 initialize해준다. 그리고 우리는 (1,1)좌표에서 출발해서 가장 먼저 dir=1인 이동을 할 것이므로 이를 doubly linked list의 pointer인 head에 insert해준다.

이후 doubly linked list가 비어있지 않고 아직 출구에 도달하지 못했을 동안

1. doubly linked list에서 head->rlink 값을 제거하여 이의 이동 정보(position)를 저장한다.
2. 아직 position에서 이동하지 않은 방향이 있거나 출구에 아직 도달하지 못한 동안

2-1. 이동정보를 통해 다음 좌표를 계산하고 다음 좌표가 출구라면 flag인 found를 1로 바꾸고 아니라면 다음 좌표의 미로가 이동할 수 있고 경로 상에서 이동하지 않은 곳인지 체크한다.

2-1-1. 위의 조건을 만족한다면(이동할 수 있는 위치라면), 해당 좌표로 이동한 것으로 mark의 값을 바꿔주고 doubly linked list에 이동정보를 insert해주고 현재 좌표를 저장했던 다음 좌표로 변경한 후 다음 좌표에서는 0번 dir 부터 다시 체크하므로 dir을 0으로 바꿔준다.

2-1-2. 위의 조건을 만족하지 못했다면(이동할 수 없는 위치라면) 다음 direction을 검토하기 위해 dir에 1을 더해준다.

위의 while loop이 끝나면 path가 발견됐는지 확인한다.

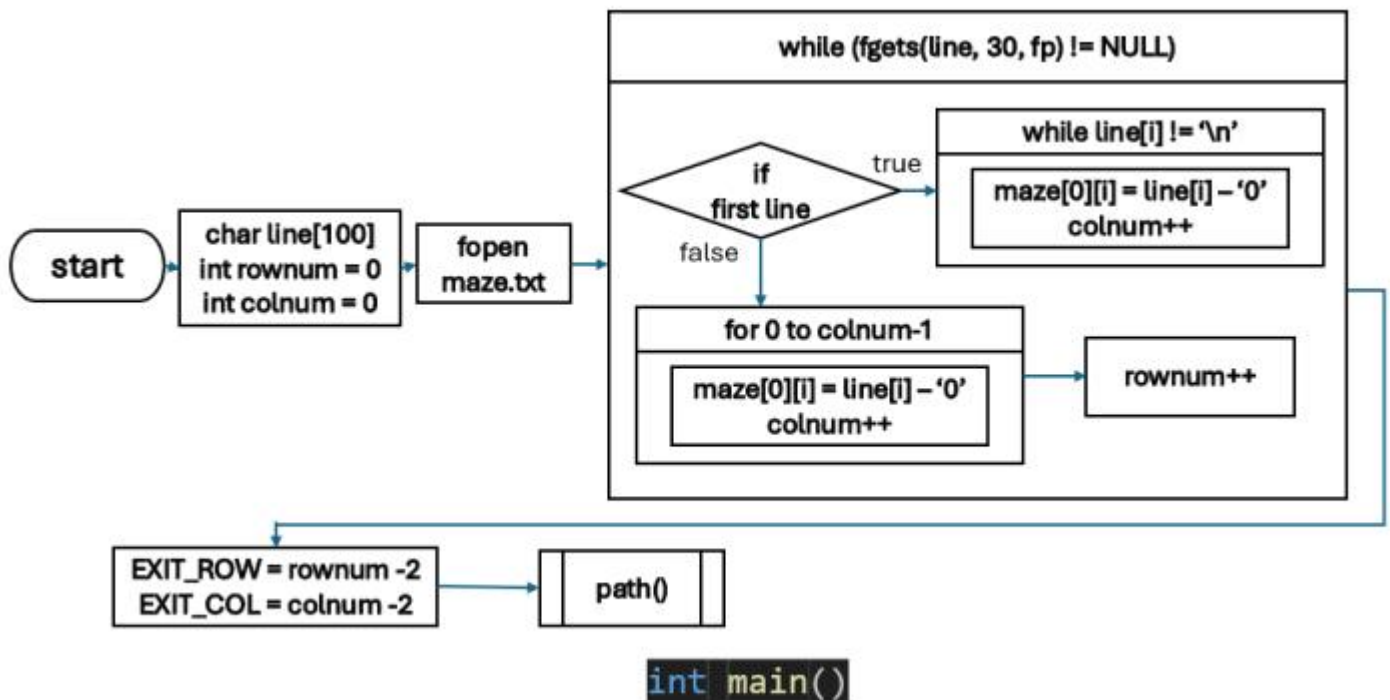
1. path를 찾았다면

path.txt를 열어 head의 llink부터 하나씩 이동하면서 각 row col을 한 줄씩 출력한다.

llink가 head라면 for loop에서 벗어나고 doubly linked list에서 아직 출력하지 않은 마지막 row col을 한 줄 출력해 준다.

이후 미로 이동에서 출구 도달 직전 좌표를 한 줄 출력해주고 마지막으로 출구의 좌표를 출력하여 파일을 닫고 끝낸다.

2. path를 못 찾았다면 path가 없다고 메시지를 출력한다.



main 함수는 가장 먼저 fgets의 buffer로 사용할 string line, 열 개수를 셀 rownum, 행 개수를 셀 colnum을 declare한다.

이후 maze.txt를 열어 첫번째 줄을 읽어 이를 maze에 int값으로 저장하면서 colnum을 저장한다.

이후 다음줄 부터는 colnum만큼 줄에서 읽으면서 미로의 값을 maze에 저장하고 모두 읽으면 rownum을 저장한다.

이후 출구의 좌표를 각 열과 행 개수에서 2만큼 뺀 값으로 저장하고 path()를 call한다.

이는 제시된 출력 예시를 만족한다.

이는 제시된 출력 예시를 만족한다.